Infinite Aura - Phase 2 Handoff Document

For: Verdent Desktop (Claude Opus 4.5)
From: Phase 1 Completion (Claude Sonnet 4)
Date: January 6, 2026
Status: Phase 1 Complete ✅ → Phase 2 Ready 🚀

Part 1: Phase 1 Completion Summary

What Was Built (Phase 1):

Phase 1 successfully established the **foundational infrastructure** for Infinite Aura's memory system:

1. **Exception Hierarchy** (14 classes)
   - Base exceptions with context tracking
   - Domain-specific exceptions (memory, security, guardrails)
   - Rich error messages with actionable guidance

2. **Guardrails Foundation** (7 policies, 8 interfaces)
   - Policy types: BLOCK, WARN, AUDIT, REQUIRE_APPROVAL
   - Append-only enforcement for history/
   - Path traversal prevention
   - Sensitive data protection

3. **Memory Scaffold** (6 classes, 18 UFC directories)
   - PathValidator - Validates and normalizes paths
   - FileOperations - Safe file read/write/append
   - DirectoryOperations - Safe directory creation
   - MemoryScaffold - Orchestrates memory operations
   - SecurityAuditLogger - Logs all security events
   - FileNamingConvention - YYYY-MM-DD_HHmmss_TYPE_description.ext

4. **Security Hardening**
   - Audit logging for all file operations
   - File locking for concurrent access
   - Symlink protection (block all symlinks)
   - Path traversal prevention

5. **Comprehensive Tests** (472 tests, 90.05% coverage)
   - Unit tests for all components
   - Integration tests for memory scaffold
   - Security tests for attack vectors
   - All tests passing ✅

6. **Complete Documentation**
   - API documentation
   - Architecture diagrams

- Verification procedures
- Living Hydration document

Key Components Available:

```
src/
├── exceptions/
│   ├── base-exceptions.ts          # BaseInfiniteAuraError, ErrorContext
│   ├── memory-exceptions.ts        # MemoryOperationError, InvalidPathError, etc.
│   ├── security-exceptions.ts      # SecurityViolationError, AuditLogError
│   └── guardrail-exceptions.ts     # GuardrailViolationError, PolicyError
├── guardrails/
│   ├── types.ts                    # GuardrailPolicy, PolicyType, GuardrailContext
│   ├── policies/                   # 7 policy implementations
│   └── validator.ts                # GuardrailValidator
├── memory/
│   ├── path-validator.ts           # PathValidator
│   ├── file-operations.ts          # FileOperations
│   ├── directory-operations.ts     # DirectoryOperations
│   ├── memory-scaffold.ts          # MemoryScaffold (orchestrator)
│   ├── security-audit-logger.ts    # SecurityAuditLogger
│   └── file-naming-convention.ts   # FileNamingConvention
tests/
├── exceptions/                     # 89 tests
├── guardrails/                     # 147 tests
└── memory/                         # 236 tests
```

APIs Available for Phase 2:

Memory Scaffold APIs:

```
// Initialize UFC structure
await MemoryScaffold.initialize(basePath: string): Promise<void>

// File operations
await FileOperations.readFile(filePath: string): Promise<string>
await FileOperations.writeFile(filePath: string, content: string): Promise<void>
await FileOperations.appendFile(filePath: string, content: string): Promise<void>

// Directory operations
await DirectoryOperations.createDirectory(dirPath: string): Promise<void>
await DirectoryOperations.ensureDirectory(dirPath: string): Promise<void>

// Path validation
PathValidator.validate(filePath: string, basePath: string): string
PathValidator.isWithinBasePath(filePath: string, basePath: string): boolean

// Security audit
await SecurityAuditLogger.logSecurityEvent(
  event: string,
  severity: 'low' | 'medium' | 'high' | 'critical',
  details: Record<string, any>
): Promise<void>

// File naming
FileNamingConvention.generate(type: string, description: string): string
FileNamingConvention.parse(filename: string): { timestamp, type, description, exten-
sion }
```

UFC Directory Structure (Ready for Phase 2):

```
.ufc/
├── user/                    # User-level context
│   ├── profile/
│   ├── preferences/
│   └── history/             # APPEND-ONLY (guardrail enforced)
├── projects/                # Project-level context
│   ├── active/
│   ├── archived/
│   └── templates/
├── sessions/                # Session-level context
│   ├── current/
│   └── history/             # APPEND-ONLY
├── agents/                  # Agent-level context
│   ├── configs/
│   ├── learned/             # High-signal events promoted here
│   └── history/             # APPEND-ONLY
└── global/                  # Global shared context
    ├── knowledge_base/
    ├── patterns/
    └── audit/               # Security audit logs
```

Quality Metrics (Phase 1):

Metric	Value	Status
Tests Passing	472/472	✅
Code Coverage	90.05%	✅
Branches Covered	87.2%	✅
Functions Covered	92.1%	✅
Lines Covered	90.05%	✅
TypeScript Strict	Enabled	✅
ESLint Violations	0	✅
Prettier Format	All files	✅

Phase 1 Accomplishments:

✅ **Memory scaffold operational** - Can safely read/write/append files
✅ **Guardrails enforced** - All security policies active
✅ **Security hardened** - Audit logs, file locks, symlink protection
✅ **UFC structure initialized** - 18 directories ready
✅ **Exception hierarchy complete** - Rich error handling
✅ **Comprehensive tests** - 472 tests, 90.05% coverage
✅ **Documentation complete** - API, architecture, verification
✅ **Git initialized** - Version control ready

Part 2: Phase 2 Overview

Phase 2 Goal:

Build the Hook System, Dynamic Context Loading, and Content Routing

Phase 2 transforms Infinite Aura from a **static memory scaffold** into an **intelligent, event-driven context system** that:
- Captures events in real-time (hook system)
- Routes events to correct directories (content-based routing)
- Identifies high-signal events (interestingness scoring)
- Loads context dynamically based on AI-driven relevance (not static 4-layer)
- Supports multiple hydration strategies per task type

Phase 2 Capabilities (5 total):

1. Hook System 🎯

Event-driven architecture for capturing and processing events.

Components:
- EventEmitter - Pub/sub pattern for event distribution
- HookHandler - Base class for all hook handlers
- 4 hook handlers:
- `capture-all-handler` - Captures all events to history/
- `stop-handler` - Captures stop events with termination reason
- `subagent-stop-handler` - Captures subagent completion events
- `session-summary-handler` - Generates session summaries

Events flow:

```
Event occurs → EventEmitter → HookHandlers → ContentRouter → Memory Scaffold
```

2. Content-Based Routing 🧭

Routes events to correct UFC directories based on event content (not just type).

Classification Logic:
- **Agent type** → routes to `.ufc/agents/{agent_id}/`
- **Project context** → routes to `.ufc/projects/{project_id}/`
- **Task type** → routes to appropriate subdirectory
- **File operations** → routes to relevant project directory
- **Learning events** → routes to `.ufc/agents/learned/`

Example:

```
Event: "DeepResearchAgent completed analysis"
→ Classification: agent_event, research_task
→ Route to: .ufc/agents/deep_research/history/
→ Also tag: #research, #analysis, #complete
```

3. Learning Indicators / Interestingness Scoring 💡

Tags high-signal events that represent learning opportunities.

Scoring Criteria:
- **Decisions** (0.7) - Agent made a significant decision
- **Breakthroughs** (0.9) - Agent discovered a novel solution
- **Failures** (0.8) - Agent encountered a failure (high learning value)
- **Patterns** (0.6) - Agent identified a recurring pattern
- **Edge cases** (0.7) - Agent handled an edge case

Promotion to learned/:

```
Event with interestingness >= 0.7 → Promoted to .ufc/agents/learned/
```

4. Dynamic Context Loading 🧠

AI-driven context selection (not static 4-layer).

Traditional Approach (Static):

```
Always load: User → Project → Session → Agent
Problem: Loads irrelevant context, slow, fixed order
```

Infinite Aura Approach (Dynamic):

```
AI analyzes current task → Selects relevant context → Loads only what's needed
Benefits: Faster, more relevant, adapts to task type
```

Example:

```
Task: "Fix bug in authentication module"
AI selects:
- User preferences (for coding style)
- Current project (for codebase context)
- Recent authentication-related events
- NOT: Old project history, unrelated agents
```

5. Hydration Strategy Registry 📋

Named strategies for different task types.

Strategy Examples:
- `deep_research` - Load extensive context, historical patterns
- `quick_fix` - Load minimal context, recent errors only
- `build_spec` - Load project templates, similar specs
- `code_review` - Load style guides, previous reviews
- `exploration` - Load user preferences, global knowledge base

Strategy Selection:

```
const strategy = HydrationStrategyRegistry.getStrategy(taskType);
const context = await strategy.loadContext(contextLoader, taskContext);
```

Phase 2 Deliverables:

Deliverable	Description	Tests
Hook system architecture	EventEmitter + 4 hook handlers	30+ tests
Content router	Classification logic + routing	25+ tests
Interestingness scorer	Learning indicators + scoring	20+ tests
Dynamic context loader	AI-driven context selection	30+ tests
Hydration strategy registry	Named strategies + selection	15+ tests
Total	**5 major components**	**120+ tests**

Estimated Duration:

Total: 12-18 hours (5 prompts: PROMPT 8-12)

Prompt	Component	Duration	Critical Level
PROMPT 8	Hook System	2-3 hours	🔴 HIGH
PROMPT 9	Content Routing	2-3 hours	🟡 MEDIUM
PROMPT 10	Interestingness Scoring	2-3 hours	🟡 MEDIUM
PROMPT 11	Dynamic Context Loading	3-5 hours	🔴 HIGH
PROMPT 12	Hydration Strategy Registry	2-3 hours	🟢 LOW

Phase 2 Success Criteria:

Overall:
- ✅ All 5 capabilities implemented
- ✅ 550+ total tests passing (472 Phase 1 + 120 Phase 2)
- ✅ Coverage maintained >85% (target: >88%)
- ✅ All quality gates passing
- ✅ Integration with Phase 1 working seamlessly

Per-Capability:
- ✅ Hook system captures events correctly
- ✅ Content router routes to correct directories
- ✅ Interestingness scorer identifies high-signal events
- ✅ Dynamic context loader selects relevant context
- ✅ Hydration strategies work for different task types

Part 3: Integration Points with Phase 1

How Phase 2 Connects to Phase 1:

Phase 2 **builds on top of** Phase 1's memory scaffold. All Phase 2 components use Phase 1 APIs.

Hook System → Memory Scaffold

Connection:

```
// Hook handlers write to memory using Phase 1 APIs
class CaptureAllHandler extends HookHandler {
  async handle(event: Event): Promise<void> {
    const content = JSON.stringify(event) + '\n'; // JSONL format
    const filePath = this.getEventFilePath(event);

    // Uses Phase 1 FileOperations API
    await FileOperations.appendFile(filePath, content);
  }
}
```

Phase 1 APIs used:

- ✅ `FileOperations.appendFile()` - Append events to history files
- ✅ `FileOperations.writeFile()` - Write session summaries
- ✅ `DirectoryOperations.ensureDirectory()` - Ensure event directories exist
- ✅ `SecurityAuditLogger.logSecurityEvent()` - Log all hook events

Integration points:

1. All events written to `.ufc/*/history/` directories
2. Guardrails enforce APPEND-ONLY for history/
3. Security audit logs all hook operations
4. File naming follows Phase 1 convention

Content Router → Memory Scaffold

Connection:

```
class ContentRouter {
  async routeEvent(event: Event): Promise<string> {
    const targetDir = this.classifyEvent(event);

    // Uses Phase 1 DirectoryOperations API
    await DirectoryOperations.ensureDirectory(targetDir);

    return targetDir;
  }
}
```

Phase 1 APIs used:

- ✅ `DirectoryOperations.ensureDirectory()` - Ensure target directories exist
- ✅ `PathValidator.validate()` - Validate all target paths
- ✅ `PathValidator.isWithinBasePath()` - Ensure paths within .ufc/

Integration points:

1. Router validates all paths before writing

2. Router respects guardrail policies
3. Router logs all routing decisions to audit log

Content Router → Guardrails

Connection:

```
class ContentRouter {
  async routeEvent(event: Event): Promise<string> {
    const targetPath = this.getTargetPath(event);

    // Check guardrail policies before routing
    const context: GuardrailContext = {
      filePath: targetPath,
      operation: 'append',
      content: JSON.stringify(event),
    };

    // Uses Phase 1 GuardrailValidator API
    await GuardrailValidator.validate(context, this.policies);

    return targetPath;
  }
}
```

Phase 1 APIs used:
- ✅ `GuardrailValidator.validate()` - Validate before routing
- ✅ Append-only policy for history/ directories
- ✅ Path traversal prevention
- ✅ Sensitive data detection

Integration points:
1. All routing respects guardrail policies
2. Violations logged to security audit
3. BLOCK policy prevents routing to forbidden paths

Interestingness Scorer → Memory Scaffold

Connection:

```
class InterestingnessScorer {
  async promoteToLearned(event: Event): Promise<void> {
    const score = this.scoreEvent(event);

    if (score >= this.threshold) {
      const learnedPath = '.ufc/agents/learned/';
      const filename = FileNamingConvention.generate(
        'learned',
        event.metadata.description || 'event'
      );

      // Uses Phase 1 FileOperations API
      await FileOperations.writeFile(
        `${learnedPath}${filename}`,
        JSON.stringify(event, null, 2)
      );
    }
  }
}
```

Phase 1 APIs used:

- ✅ `FileOperations.writeFile()` - Write to learned/
- ✅ `FileNamingConvention.generate()` - Generate learned item filename
- ✅ `SecurityAuditLogger.logSecurityEvent()` - Log promotions

Integration points:

1. High-signal events promoted to learned/
2. File naming follows Phase 1 convention
3. All promotions logged to audit

Dynamic Context Loader → Memory Scaffold

Connection:

```
class DynamicContextLoader {
  async loadContext(taskContext: TaskContext): Promise<ContextData> {
    // AI selects relevant context files
    const relevantFiles = await this.selectRelevantFiles(taskContext);

    const contextData: ContextData = {
      user: null,
      project: null,
      session: null,
      agent: null,
    };

    // Uses Phase 1 FileOperations API to read context
    for (const file of relevantFiles) {
      const content = await FileOperations.readFile(file.path);

      // Parse and add to contextData
      this.addToContext(contextData, file.layer, content);
    }

    return contextData;
  }
}
```

Phase 1 APIs used:
- ✅ `FileOperations.readFile()` - Read context files
- ✅ `PathValidator.validate()` - Validate context file paths
- ✅ `DirectoryOperations.ensureDirectory()` - Ensure context directories exist

Integration points:
1. Reads from all UFC directories (user/, projects/, sessions/, agents/)
2. Loads 4-layer context (User → Project → Session → Agent)
3. Validates all paths before reading
4. Caches frequently-accessed context

Integration Summary

```
Phase 2 Components → Phase 1 APIs
═══════════════════════════════════════════════════

Hook System          → FileOperations.appendFile()
                     → DirectoryOperations.ensureDirectory()
                     → SecurityAuditLogger.logSecurityEvent()

Content Router       → DirectoryOperations.ensureDirectory()
                     → PathValidator.validate()
                     → GuardrailValidator.validate()

Interestingness      → FileOperations.writeFile()
Scorer               → FileNamingConvention.generate()
                     → SecurityAuditLogger.logSecurityEvent()

Dynamic Context      → FileOperations.readFile()
Loader               → PathValidator.validate()
                     → DirectoryOperations.ensureDirectory()

Hydration Strategy   → DynamicContextLoader (which uses Phase 1 APIs)
Registry
```

Part 4: PAI Patterns to Adapt

From Daniel's PAI Repository:

Daniel's PAI (Personal AI) repository provides proven patterns for building context-aware AI systems. Infinite Aura adapts these patterns to TypeScript and extends them with more intelligent routing and scoring.

Hook System Patterns

PAI Implementation (Bun):

```
// capture-all-events.ts
export default {
  onEvent: async (event: any) => {
    await Bun.write(`${ufc_path}/history/events.jsonl`, JSON.stringify(event) + '\n',
{
      append: true
    });
  }
}

// stop-hook.ts
export default {
  onStop: async (reason: string) => {
    await Bun.write(`${ufc_path}/sessions/current/stop.json`, JSON.stringify({
reason }));
  }
}
```

Infinite Aura Adaptation (TypeScript):

```
// event-emitter.ts
class EventEmitter {
  private handlers: Map<EventType, HookHandler[]> = new Map();

  async emit(event: Event): Promise<void> {
    const handlers = this.handlers.get(event.type) || [];
    await Promise.all(handlers.map(h => h.handle(event)));
  }
}

// capture-all-handler.ts
class CaptureAllHandler extends HookHandler {
  async handle(event: Event): Promise<void> {
    const content = JSON.stringify(event) + '\n';
    await FileOperations.appendFile(this.getEventFilePath(event), content);
  }
}
```

Key Differences:
- ✅ Bun → Node.js/TypeScript (cross-platform)
- ✅ Direct file writes → Phase 1 FileOperations (safe, validated)
- ✅ Simple callbacks → Event emitter pattern (scalable)
- ✅ Synchronous → Fully async with error handling

JSONL Event Format

PAI Format:

```
{"timestamp":"2024-01-01T12:00:00Z","type":"message","content":"User mes-
sage","metadata":{}}
{"timestamp":"2024-01-01T12:00:01Z","type":"response","content":"AI response","metadat
a":{}}
```

Infinite Aura Format (Same + Extensions):

```
{"timestamp":"2026-01-06T12:00:00Z","type":"capture_all","content":"Event con-
tent","metadata":{"agentId":"deep_research","projectId":"proj_123","sessionId":"sess_4
56","tags":["research","analysis"],"interestingness":0.85}}
{"timestamp":"2026-01-06T12:00:01Z","type":"stop","content":"Task completed","metadata
":{"reason":"success","duration":3600}}
```


TypeScript Interfaces:

```typescript
interface Event {
  timestamp: string; // ISO 8601
  type: EventType;
  content: string;
  metadata: EventMetadata;
}

interface EventMetadata {
  agentId?: string;
  projectId?: string;
  sessionId?: string;
  tags?: string[];
  interestingness?: number; // 0-1 score
  [key: string]: any; // Extensible
}

enum EventType {
  CAPTURE_ALL = 'capture_all',
  STOP = 'stop',
  SUBAGENT_STOP = 'subagent_stop',
  SESSION_SUMMARY = 'session_summary',
}
```

Directory Routing

PAI Approach (Type-based):

```typescript
// Simple type-based routing
function routeEvent(event: Event): string {
  switch (event.type) {
    case 'message':
      return `${ufc_path}/history/messages.jsonl`;
    case 'session_summary':
      return `${ufc_path}/sessions/current/summary.json`;
    default:
      return `${ufc_path}/history/events.jsonl`;
  }
}
```

Infinite Aura Approach (Content-based):

```
// Intelligent content-based routing
class ContentRouter {
  classifyEvent(event: Event): string {
    // Analyze event content, not just type
    const classification = {
      agentType: this.extractAgentType(event),
      taskType: this.extractTaskType(event),
      projectContext: this.extractProjectContext(event),
      tags: this.extractTags(event),
    };

    // Route based on multiple factors
    if (classification.agentType === 'deep_research') {
      return `.ufc/agents/deep_research/history/`;
    } else if (classification.projectContext) {
      return `.ufc/projects/${classification.projectContext}/history/`;
    } else {
      return `.ufc/sessions/current/history/`;
    }
  }
}
```

Key Differences:
- ❌ Type-based → ✅ Content-based (more intelligent)
- ❌ Simple switch → ✅ Classification logic
- ❌ Fixed routing → ✅ Context-aware routing

Context Loading

PAI Approach (Static 4-layer):

```
// Always load all 4 layers in fixed order
async function loadContext(): Promise<Context> {
  const user = await loadUserContext();
  const project = await loadProjectContext();
  const session = await loadSessionContext();
  const agent = await loadAgentContext();

  return { user, project, session, agent };
}
```

Infinite Aura Approach (Dynamic AI-driven):

```
// AI selects relevant context based on task
class DynamicContextLoader {
  async loadContext(taskContext: TaskContext): Promise<ContextData> {
    // AI analyzes task and selects relevant context
    const relevantFiles = await this.selectRelevantFiles(taskContext);

    // Load only what's needed
    const contextData: ContextData = {
      user: null,
      project: null,
      session: null,
      agent: null,
    };

    for (const file of relevantFiles) {
      const content = await FileOperations.readFile(file.path);
      this.addToContext(contextData, file.layer, content);
    }

    return contextData;
  }

  private async selectRelevantFiles(taskContext: TaskContext): Promise<ContextFile[]>
{
    // Use LLM to select relevant files
    const prompt = `Given task: ${taskContext.description}, select relevant context
files from:
    - User: ${this.listUserFiles()}
    - Project: ${this.listProjectFiles()}
    - Session: ${this.listSessionFiles()}
    - Agent: ${this.listAgentFiles()}`;

    const response = await this.llmClient.complete(prompt);
    return this.parseFileSelection(response);
  }
}
```

Key Differences:
- ❌ Static → ✅ Dynamic (AI-driven)
- ❌ Always 4 layers → ✅ Load only relevant layers
- ❌ Fixed order → ✅ Relevance-based ordering
- ❌ Load everything → ✅ Load what's needed (faster)

File Naming

PAI Convention:

```
2024-01-01_120000_TYPE_description.ext
```

Infinite Aura Convention (Already Implemented in Phase 1):

```
// FileNamingConvention class from Phase 1
FileNamingConvention.generate('event', 'user_message');
// → 2026-01-06_120000_EVENT_user_message.jsonl

FileNamingConvention.generate('learned', 'authentication_pattern');
// → 2026-01-06_120000_LEARNED_authentication_pattern.json
```

Reuse Existing Class:

```
// Phase 2 uses Phase 1 FileNamingConvention directly
import { FileNamingConvention } from '../memory/file-naming-convention';

// In hook handlers
const filename = FileNamingConvention.generate('capture_all', 'event');

// In interestingness scorer
const filename = FileNamingConvention.generate('learned', event.metadata.description);
```

Adaptation Summary

PAI Pattern	Infinite Aura Adaptation	Status
Bun hooks	TypeScript event emitter + handlers	✅ PROMPT 8
JSONL format	Same + extensions (interest-ingness, tags)	✅ PROMPT 8
Type-based routing	Content-based routing (more intelligent)	✅ PROMPT 9
Static 4-layer	Dynamic AI-driven context loading	✅ PROMPT 11
File naming	Reuse Phase 1 FileNaming-Convention	✅ Phase 1

Part 5: Prompt Sequence (PROMPT 8-12)

PROMPT 8: Hook System Foundation

Goal: Create event emitter and basic hook handlers

Duration: 2-3 hours

Critical Level: 🔴 **HIGH** (mark for Claude Code review)

Deliverables:

1. **EventEmitter class** (`src/hooks/event-emitter.ts`)
   - Pub/sub pattern for event distribution
   - Register/unregister handlers
   - Emit events to all registered handlers
   - Error handling for handler failures
   - Async event emission

2. **Event interface & EventType enum** (`src/hooks/types.ts`)
   - Event interface (timestamp, type, content, metadata)

   - EventType enum (CAPTURE_ALL, STOP, SUBAGENT_STOP, SESSION_SUMMARY)
   - EventMetadata interface (agentId, projectId, sessionId, tags, interestingness)

3. **HookHandler base class** (`src/hooks/hook-handler.ts`)
   - Abstract base class for all hook handlers
   - Common methods: handle(), getEventFilePath(), validate()
   - Error handling and logging

4. **4 hook handlers:**
   - `capture-all-handler.ts` - Captures all events to history/
   - `stop-handler.ts` - Captures stop events with termination reason
   - `subagent-stop-handler.ts` - Captures subagent completion events
   - `session-summary-handler.ts` - Generates session summaries

5. **Tests for hook system** (`tests/hooks/`)
   - EventEmitter tests (30+ tests)
   - HookHandler tests
   - Integration tests with Phase 1 FileOperations

Success Criteria:

✅ EventEmitter class created and tested
✅ Event interface and EventType enum defined
✅ HookHandler base class created
✅ 4 hook handlers implemented and tested
✅ Events captured and written to memory using Phase 1 APIs
✅ All tests passing (30+ tests)
✅ Coverage >85% for hooks/
✅ Integration with Phase 1 FileOperations working

Critical Aspects (Claude Code Review):

🔴 **Event emitter design:**
- Pub/sub pattern implementation
- Handler registration/unregistration
- Error handling (one handler failure shouldn't break others)
- Async event emission (Promise.all vs sequential)

🔴 **Hook handler lifecycle:**
- Initialization
- Event validation
- Error handling
- Cleanup

🔴 **Performance considerations:**
- Async hooks (don't block main thread)
- Event queue management (what if events come faster than handlers can process?)
- Memory leaks (handler cleanup)

Implementation Notes:

```
// src/hooks/event-emitter.ts
export class EventEmitter {
  private handlers: Map<EventType, HookHandler[]> = new Map();

  registerHandler(type: EventType, handler: HookHandler): void {
    const handlers = this.handlers.get(type) || [];
    handlers.push(handler);
    this.handlers.set(type, handlers);
  }

  async emit(event: Event): Promise<void> {
    const allHandlers = this.handlers.get(event.type) || [];
    const wildcardHandlers = this.handlers.get(EventType.CAPTURE_ALL) || [];

    // Run all handlers in parallel (don't let one failure block others)
    const results = await Promise.allSettled([
      ...allHandlers.map(h => h.handle(event)),
      ...wildcardHandlers.map(h => h.handle(event)),
    ]);

    // Log any handler failures
    results.forEach((result, index) => {
      if (result.status === 'rejected') {
        console.error(`Handler ${index} failed:`, result.reason);
      }
    });
  }
}

// src/hooks/hook-handler.ts
export abstract class HookHandler {
  abstract handle(event: Event): Promise<void>;

  protected getEventFilePath(event: Event): string {
    // Use Phase 1 FileNamingConvention
    const filename = FileNamingConvention.generate(
      event.type,
      event.metadata.description || 'event'
    );
    return `${this.basePath}/${filename}`;
  }
}
```

PROMPT 9: Content-Based Routing

Goal: Create content router and classification logic

Duration: 2-3 hours

Critical Level: 🟡 **MEDIUM**

Deliverables:

1. **ContentRouter class** (`src/routing/content-router.ts`)
   - Classification logic (agent type, task type, file path, tags)
   - Route events to correct directories

   - Integration with Phase 1 DirectoryOperations
   - Path validation before routing

2. **Classification logic:**
   - Extract agent type from event content
   - Extract task type (research, coding, debugging, etc.)
   - Extract project context
   - Extract tags (#research, #bug, #feature, etc.)

3. **Route events to correct directories:**
   - Agent events → `.ufc/agents/{agent_id}/history/`
   - Project events → `.ufc/projects/{project_id}/history/`
   - Session events → `.ufc/sessions/current/history/`
   - Learning events → `.ufc/agents/learned/`

4. **Tests for routing** (`tests/routing/`)
   - Classification tests (25+ tests)
   - Routing logic tests
   - Integration tests with Phase 1 DirectoryOperations

Success Criteria:

✅ ContentRouter class created and tested
✅ Classification logic implemented
✅ Events routed to correct directories
✅ Integration with Phase 1 DirectoryOperations working
✅ All paths validated before routing
✅ All tests passing (25+ tests)
✅ Coverage >85% for routing/

Critical Aspects (Claude Code Review):

🟡 **Classification logic:**
- How to determine agent type from event content?
- How to extract task type?
- What if event belongs to multiple categories?
- Edge cases (ambiguous events)

🟡 **Routing rules:**
- Priority order (agent > project > session)?
- Fallback routing (if classification fails)?
- Duplicate routing (event to multiple directories)?

Implementation Notes:

```
// src/routing/content-router.ts
export class ContentRouter {
  classifyEvent(event: Event): Classification {
    return {
      agentType: this.extractAgentType(event),
      taskType: this.extractTaskType(event),
      projectContext: this.extractProjectContext(event),
      tags: this.extractTags(event),
    };
  }

  async routeEvent(event: Event): Promise<string> {
    const classification = this.classifyEvent(event);

    // Determine target directory based on classification
    let targetDir: string;

    if (classification.agentType) {
      targetDir = `.ufc/agents/${classification.agentType}/history/`;
    } else if (classification.projectContext) {
      targetDir = `.ufc/projects/${classification.projectContext}/history/`;
    } else {
      targetDir = `.ufc/sessions/current/history/`;
    }

    // Ensure directory exists (Phase 1 API)
    await DirectoryOperations.ensureDirectory(targetDir);

    // Validate path (Phase 1 API)
    PathValidator.validate(targetDir, this.basePath);

    return targetDir;
  }

  private extractAgentType(event: Event): string | null {
    // Look for agent identifiers in event content
    const agentPatterns = [
      /DeepResearchAgent/i,
      /CodeReviewAgent/i,
      /BuildSpecAgent/i,
    ];

    for (const pattern of agentPatterns) {
      if (pattern.test(event.content)) {
        return pattern.source.replace(/Agent|\/i/g, '').toLowerCase();
      }
    }

    return event.metadata.agentId || null;
  }
}
```

PROMPT 10: Learning Indicators & Interestingness Scoring

Goal: Create interestingness scorer and learning indicators

Duration: 2-3 hours

Critical Level: 🟡 MEDIUM

Deliverables:

1. **InterestingnessScorer class** (`src/routing/interestingness-scorer.ts`)
   - Scoring logic (decisions, breakthroughs, failures, patterns)
   - Score calculation (0-1 scale)
   - Threshold configuration (default: 0.7)

2. **Scoring logic:**
   - **Decisions** (0.7) - Agent made a significant decision
   - **Breakthroughs** (0.9) - Agent discovered a novel solution
   - **Failures** (0.8) - Agent encountered a failure (high learning value)
   - **Patterns** (0.6) - Agent identified a recurring pattern
   - **Edge cases** (0.7) - Agent handled an edge case

3. **Tag high-signal events:**
   - Add `interestingness` score to event metadata
   - Add learning indicators (#decision, #breakthrough, #failure, etc.)

4. **Promote to learned/:**
   - Events with `interestingness >= 0.7` promoted to `.ufc/agents/learned/`
   - Use Phase 1 FileOperations to write learned items

5. **Tests for scoring** (`tests/routing/`)
   - Scoring logic tests (20+ tests)
   - Promotion logic tests
   - Integration tests with Phase 1 FileOperations

Success Criteria:

✅ InterestingnessScorer class created and tested
✅ Scoring logic implemented (5 criteria)
✅ High-signal events tagged with interestingness score
✅ Events promoted to learned/ when score >= threshold
✅ All tests passing (20+ tests)
✅ Coverage >85% for interestingness-scorer.ts

Critical Aspects (Claude Code Review):

🟡 **Scoring algorithm:**
- What makes an event “interesting”?
- How to combine multiple indicators?
- Threshold tuning (too high = miss learning, too low = noise)

🟡 **Learning indicators:**
- How to detect decisions vs regular events?
- How to detect breakthroughs vs incremental progress?
- How to detect patterns?

Implementation Notes:

```typescript
// src/routing/interestingness-scorer.ts
export class InterestingnessScorer {
  private readonly threshold = 0.7;

  scoreEvent(event: Event): number {
    let score = 0;

    // Check for decision indicators
    if (this.isDecision(event)) score = Math.max(score, 0.7);

    // Check for breakthrough indicators
    if (this.isBreakthrough(event)) score = Math.max(score, 0.9);

    // Check for failure indicators (high learning value)
    if (this.isFailure(event)) score = Math.max(score, 0.8);

    // Check for pattern indicators
    if (this.isPattern(event)) score = Math.max(score, 0.6);

    // Check for edge case indicators
    if (this.isEdgeCase(event)) score = Math.max(score, 0.7);

    return score;
  }

  async promoteToLearned(event: Event): Promise<void> {
    const score = this.scoreEvent(event);

    if (score >= this.threshold) {
      // Add interestingness score to metadata
      event.metadata.interestingness = score;

      // Generate filename (Phase 1 API)
      const filename = FileNamingConvention.generate(
        'learned',
        event.metadata.description || 'event'
      );

      const learnedPath = `.ufc/agents/learned/${filename}`;

      // Write to learned/ (Phase 1 API)
      await FileOperations.writeFile(
        learnedPath,
        JSON.stringify(event, null, 2)
      );

      // Log promotion (Phase 1 API)
      await SecurityAuditLogger.logSecurityEvent(
        'event_promoted_to_learned',
        'low',
        { eventType: event.type, score, path: learnedPath }
      );
    }
  }

  private isDecision(event: Event): boolean {
    const decisionKeywords = ['decided', 'chose', 'selected', 'opted for'];
    return decisionKeywords.some(k => event.content.toLowerCase().includes(k));
  }

  private isBreakthrough(event: Event): boolean {
    const breakthroughKeywords = ['breakthrough', 'discovered', 'novel solution'];
```

```
    return breakthroughKeywords.some(k => event.content.toLowerCase().includes(k));
  }
}
```

PROMPT 11: Dynamic Context Loading

Goal: Create AI-driven context loader (not static)

Duration: 3-5 hours

Critical Level: 🔴 **HIGH** (mark for Claude Code review)

Deliverables:

1. **DynamicContextLoader class** (`src/context/dynamic-context-loader.ts`)
   - AI-driven context selection (using LLM API)
   - Relevance scoring for context files
   - 4-layer context loading (User → Project → Session → Agent)
   - Context caching for performance

2. **AI-driven context selection:**
   - Use LLM to analyze task and select relevant context files
   - Prompt engineering for context selection
   - Parse LLM response to extract file paths

3. **Relevance scoring:**
   - Score each context file for relevance to current task
   - Load high-relevance files first
   - Skip low-relevance files to save time

4. **4-layer context loading:**
   - User layer: Load from `.ufc/user/`
   - Project layer: Load from `.ufc/projects/{project_id}/`
   - Session layer: Load from `.ufc/sessions/current/`
   - Agent layer: Load from `.ufc/agents/{agent_id}/`

5. **Tests for context loading** (`tests/context/`)
   - Context selection tests (30+ tests)
   - Relevance scoring tests
   - Integration tests with Phase 1 FileOperations
   - Mock LLM responses for testing

Success Criteria:

✅ DynamicContextLoader class created and tested
✅ AI-driven context selection working
✅ Relevance scoring implemented
✅ 4-layer context loading working
✅ Context caching implemented
✅ All tests passing (30+ tests)
✅ Coverage >85% for context/
✅ Integration with Phase 1 FileOperations working

Critical Aspects (Claude Code Review):

🔴 **AI-driven context selection:**
- LLM API integration (Anthropic Claude API)
- Prompt engineering (how to ask LLM to select context?)
- Response parsing (extract file paths from LLM response)
- Error handling (what if LLM API fails?)

🔴 **Relevance scoring algorithm:**
- How to score relevance?
- What features to consider? (recency, frequency, similarity)
- Threshold for loading (load all above X score?)

🔴 **Context caching strategy:**
- What to cache? (frequently-accessed files)
- Cache invalidation (when to refresh?)
- Cache size limits (don’t cache everything)

🔴 **Performance optimization:**
- Parallel file loading (don’t load sequentially)
- Lazy loading (load on-demand)
- Streaming (for large context files)

Implementation Notes:

```typescript
// src/context/dynamic-context-loader.ts
import Anthropic from '@anthropic-ai/sdk';

export class DynamicContextLoader {
  private anthropic: Anthropic;
  private cache: Map<string, ContextFile> = new Map();

  constructor(apiKey: string) {
    this.anthropic = new Anthropic({ apiKey });
  }

  async loadContext(taskContext: TaskContext): Promise<ContextData> {
    // AI selects relevant context files
    const relevantFiles = await this.selectRelevantFiles(taskContext);

    // Load context from relevant files
    const contextData: ContextData = {
      user: null,
      project: null,
      session: null,
      agent: null,
    };

    // Load files in parallel (Phase 1 API)
    await Promise.all(
      relevantFiles.map(async (file) => {
        const content = await FileOperations.readFile(file.path);
        this.addToContext(contextData, file.layer, content);
      })
    );

    return contextData;
  }

  private async selectRelevantFiles(taskContext: TaskContext): Promise<ContextFile[]>
{
    // List all available context files
    const availableFiles = await this.listAllContextFiles();

    // Use LLM to select relevant files
    const prompt = `Given this task:
    Description: ${taskContext.description}
    Type: ${taskContext.type}
    Agent: ${taskContext.agentId}

    Select the most relevant context files from this list:
    ${availableFiles.map(f => `- ${f.path}: ${f.description}`).join('\n')}

    Return a JSON array of file paths in order of relevance.`;

    const response = await this.anthropic.messages.create({
      model: 'claude-opus-4-20250514',
      max_tokens: 1024,
      messages: [{ role: 'user', content: prompt }],
    });

    const selectedPaths = this.parseFileSelection(response.content[0].text);

    // Map paths to ContextFile objects
    return availableFiles.filter(f => selectedPaths.includes(f.path));
  }
```

```
  private async listAllContextFiles(): Promise<ContextFile[]> {
    // List files from all 4 layers
    const layers = ['user', 'project', 'session', 'agent'];
    const files: ContextFile[] = [];

    for (const layer of layers) {
      const layerPath = `.ufc/${layer}/`;
      const layerFiles = await this.listFilesInDirectory(layerPath);
      files.push(...layerFiles.map(f => ({ ...f, layer })));
    }

    return files;
  }
}
```

PROMPT 12: Hydration Strategy Registry

Goal: Create named hydration strategies per task type

Duration: 2-3 hours

Critical Level: 🟢 **LOW**

Deliverables:

1. **HydrationStrategyRegistry class** (`src/context/hydration-strategy-registry.ts`)
   - Registry of named strategies
   - Strategy selection logic
   - Strategy execution

2. **Named strategies:**
   - `deep_research` - Load extensive context, historical patterns
   - `quick_fix` - Load minimal context, recent errors only
   - `build_spec` - Load project templates, similar specs
   - `code_review` - Load style guides, previous reviews
   - `exploration` - Load user preferences, global knowledge base

3. **Strategy selection logic:**
   - Select strategy based on task type
   - Fallback to default strategy if no match

4. **Tests for strategies** (`tests/context/`)
   - Strategy registration tests (15+ tests)
   - Strategy selection tests
   - Strategy execution tests
   - Integration tests with DynamicContextLoader

Success Criteria:

✅ HydrationStrategyRegistry class created and tested
✅ 5 named strategies defined
✅ Strategy selection logic implemented
✅ Strategy execution working
✅ All tests passing (15+ tests)

✅ Coverage >85% for hydration-strategy-registry.ts
✅ Integration with DynamicContextLoader working

Critical Aspects:

🟢 **Strategy definitions:**
- What context to load for each strategy?
- How to parameterize strategies?
- How to add custom strategies?

🟢 **Registry pattern:**
- How to register strategies?
- How to retrieve strategies?
- How to handle missing strategies?

Implementation Notes:

```
// src/context/hydration-strategy-registry.ts
export interface HydrationStrategy {
  name: string;
  description: string;
  execute(loader: DynamicContextLoader, taskContext: TaskContext):
Promise<ContextData>;
}

export class HydrationStrategyRegistry {
  private strategies: Map<string, HydrationStrategy> = new Map();

  registerStrategy(strategy: HydrationStrategy): void {
    this.strategies.set(strategy.name, strategy);
  }

  getStrategy(name: string): HydrationStrategy | null {
    return this.strategies.get(name) || null;
  }

  getStrategyForTaskType(taskType: string): HydrationStrategy {
    // Map task types to strategies
    const mapping: Record<string, string> = {
      'research': 'deep_research',
      'debug': 'quick_fix',
      'specification': 'build_spec',
      'review': 'code_review',
      'explore': 'exploration',
    };

    const strategyName = mapping[taskType] || 'default';
    return this.getStrategy(strategyName) || this.getStrategy('default')!;
  }
}

// Example strategy
export class DeepResearchStrategy implements HydrationStrategy {
  name = 'deep_research';
  description = 'Load extensive context for deep research tasks';

  async execute(
    loader: DynamicContextLoader,
    taskContext: TaskContext
  ): Promise<ContextData> {
    // Override context selection to load more context
    taskContext.contextDepth = 'extensive';
    taskContext.includeHistorical = true;
    taskContext.includePatterns = true;

    return await loader.loadContext(taskContext);
  }
}
```

Prompt Sequence Summary

Prompt	Component	Duration	Critical	Tests	Dependencies
PROMPT 8	Hook System	2-3h	🔴 HIGH	30+	Phase 1
PROMPT 9	Content Routing	2-3h	🟡 MEDIUM	25+	PROMPT 8
PROMPT 10	Interestingness Scoring	2-3h	🟡 MEDIUM	20+	PROMPT 9
PROMPT 11	Dynamic Context Loading	3-5h	🔴 HIGH	30+	Phase 1
PROMPT 12	Hydration Strategy Registry	2-3h	🟢 LOW	15+	PROMPT 11
TOTAL	**Phase 2**	**12-18h**	-	**120+**	-

Part 6: Critical Aspects (Claude Code Review)

🔴 HIGH PRIORITY (Must review with Claude Code)

These aspects are architecturally critical and should be reviewed by Claude Code before proceeding to the next prompt.

1. Hook System Architecture (PROMPT 8)

Why Critical:
- Foundation for all event capture
- Poor design = performance issues at scale
- Error handling affects system reliability

What to Review:

✅ **Event emitter design:**

```
// Question: Pub/sub pattern implementation
// - Should handlers be synchronous or async?
// - How to handle handler registration/unregistration?
// - What if a handler throws an error?

class EventEmitter {
  // Review: Is Map the right data structure?
  private handlers: Map<EventType, HookHandler[]> = new Map();

  // Review: Should this be Promise.all or Promise.allSettled?
  async emit(event: Event): Promise<void> {
    const handlers = this.handlers.get(event.type) || [];

    // Option 1: Fail fast (one failure stops all)
    await Promise.all(handlers.map(h => h.handle(event)));

    // Option 2: Fail gracefully (one failure doesn't affect others)
    const results = await Promise.allSettled(handlers.map(h => h.handle(event)));
  }
}
```

Decision Points:

- [ ] Use `Promise.all` (fail fast) or `Promise.allSettled` (fail gracefully)?
- [ ] Allow multiple handlers per event type or single handler?
- [ ] Support wildcard handlers (capture all events)?
- [ ] Support handler priority/ordering?

✅ **Hook handler lifecycle:**

```
// Question: Lifecycle management
// - When to initialize handlers?
// - When to cleanup handlers?
// - How to handle handler errors?

abstract class HookHandler {
  // Review: Should handlers have lifecycle methods?
  async initialize?(): Promise<void>;
  async handle(event: Event): Promise<void>;
  async cleanup?(): Promise<void>;

  // Review: How to validate events before handling?
  protected validate(event: Event): void {
    if (!event.timestamp) throw new Error('Missing timestamp');
    if (!event.type) throw new Error('Missing type');
  }
}
```

Decision Points:

- [ ] Add lifecycle methods (initialize, cleanup)?
- [ ] Validate events before handling or let handlers decide?
- [ ] Support handler state (stateful vs stateless)?

✅ **Error handling in hooks:**

```
// Question: Error handling strategy
// - What to do when a handler fails?
// - Log and continue? Retry? Fail entire event?

class EventEmitter {
  async emit(event: Event): Promise<void> {
    const results = await Promise.allSettled(
      handlers.map(h => h.handle(event))
    );

    // Review: How to handle failures?
    results.forEach((result, index) => {
      if (result.status === 'rejected') {
        // Option 1: Just log
        console.error(`Handler ${index} failed:`, result.reason);

        // Option 2: Log + record in audit
        SecurityAuditLogger.logSecurityEvent('handler_failure', 'medium', {
          handlerIndex: index,
          error: result.reason,
        });

        // Option 3: Retry
        // await this.retryHandler(handlers[index], event);
      }
    });
  }
}
```

Decision Points:

- [ ] Log and continue, or retry failed handlers?
- [ ] Record handler failures in audit log?
- [ ] Add dead letter queue for failed events?

✅ **Performance considerations:**

```
// Question: Performance at scale
// - What if events come faster than handlers can process?
// - How to prevent memory leaks?
// - How to handle slow handlers?

class EventEmitter {
  private eventQueue: Event[] = [];
  private processing = false;

  // Review: Should we queue events or process immediately?
  async emit(event: Event): Promise<void> {
    // Option 1: Process immediately (may block)
    await this.processEvent(event);

    // Option 2: Queue and process async (non-blocking)
    this.eventQueue.push(event);
    if (!this.processing) {
      this.processQueue();
    }
  }

  private async processQueue(): Promise<void> {
    this.processing = true;

    while (this.eventQueue.length > 0) {
      const event = this.eventQueue.shift()!;
      await this.processEvent(event);
    }

    this.processing = false;
  }
}
```

Decision Points:
- [ ] Process events immediately or queue them?
- [ ] Add event queue with size limits?
- [ ] Add timeout for slow handlers?
- [ ] Add handler metrics (latency, throughput)?

---

2. Dynamic Context Loading (PROMPT 11)

Why Critical:
- Core differentiator from PAI (AI-driven vs static)
- Performance impact (loading too much = slow)
- Correctness impact (loading wrong context = bad results)

What to Review:

✅ **AI-driven context selection:**

```
// Question: LLM API integration
// - Which LLM to use? (Claude Opus, GPT-4, etc.)
// - How to structure the prompt?
// - How to parse the response?

class DynamicContextLoader {
  private async selectRelevantFiles(
    taskContext: TaskContext
  ): Promise<ContextFile[]> {
    // Review: Prompt engineering
    const prompt = `Given this task:
    Description: ${taskContext.description}
    Type: ${taskContext.type}

    Select the most relevant context files from:
    ${availableFiles.map(f => `- ${f.path}: ${f.description}`).join('\n')}

    Return a JSON array of file paths in order of relevance.`;

    // Review: LLM selection
    const response = await this.anthropic.messages.create({
      model: 'claude-opus-4-20250514', // Correct model?
      max_tokens: 1024, // Enough?
      messages: [{ role: 'user', content: prompt }],
    });

    // Review: Response parsing
    const selectedPaths = this.parseFileSelection(response.content[0].text);

    return selectedPaths;
  }
}
```

Decision Points:

- [ ] Which LLM model to use? (Claude Opus 4, GPT-4, etc.)
- [ ] How many files to include in prompt? (10? 50? 100?)
- [ ] What if LLM API fails? (fallback to static loading?)
- [ ] How to parse LLM response? (JSON? natural language?)

✅ **Relevance scoring algorithm:**

```
// Question: Scoring algorithm
// - What features to consider for relevance?
// - How to combine multiple features?
// - What threshold for loading?

class DynamicContextLoader {
  private scoreRelevance(
    file: ContextFile,
    taskContext: TaskContext
  ): number {
    let score = 0;

    // Review: Feature weights
    // Recency (0-0.3)
    const ageInDays = this.getFileAge(file);
    score += Math.max(0, 0.3 - (ageInDays / 30) * 0.3);

    // Frequency (0-0.3)
    const accessCount = this.getAccessCount(file);
    score += Math.min(0.3, accessCount / 100 * 0.3);

    // Similarity (0-0.4)
    const similarity = this.computeSimilarity(file, taskContext);
    score += similarity * 0.4;

    return score;
  }
}
```

Decision Points:

- [ ] What features to consider? (recency, frequency, similarity)
- [ ] What weights for each feature?
- [ ] Use LLM for scoring or heuristic algorithm?
- [ ] What threshold for loading? (load all >0.5?)

✅ **Context caching strategy:**

```
// Question: Caching strategy
// - What to cache? (files? parsed content?)
// - When to invalidate cache?
// - How to limit cache size?

class DynamicContextLoader {
  private cache: Map<string, CachedContextFile> = new Map();
  private maxCacheSize = 100; // Review: What size?

  private async loadFile(filePath: string): Promise<string> {
    // Check cache
    const cached = this.cache.get(filePath);

    if (cached && !this.isCacheExpired(cached)) {
      return cached.content;
    }

    // Load from disk (Phase 1 API)
    const content = await FileOperations.readFile(filePath);

    // Update cache
    this.cache.set(filePath, {
      content,
      loadedAt: Date.now(),
      ttl: 60000, // Review: What TTL?
    });

    // Evict old entries if cache too large
    if (this.cache.size > this.maxCacheSize) {
      this.evictOldest();
    }

    return content;
  }
}
```

Decision Points:

- [ ] Cache files or parsed content?
- [ ] Cache size limit? (10? 100? 1000?)
- [ ] Cache TTL (time to live)? (1 min? 5 min? 1 hour?)
- [ ] Cache eviction policy? (LRU? FIFO? TTL-based?)

✅ **Performance optimization:**

```
// Question: Performance optimization
// - How to load files efficiently?
// - Parallel vs sequential loading?
// - Lazy loading?

class DynamicContextLoader {
  async loadContext(taskContext: TaskContext): Promise<ContextData> {
    const relevantFiles = await this.selectRelevantFiles(taskContext);

    // Review: Parallel vs sequential?
    // Option 1: Parallel (faster, more memory)
    const contents = await Promise.all(
      relevantFiles.map(f => FileOperations.readFile(f.path))
    );

    // Option 2: Sequential (slower, less memory)
    const contents: string[] = [];
    for (const file of relevantFiles) {
      contents.push(await FileOperations.readFile(file.path));
    }

    // Option 3: Lazy (load on-demand)
    return new LazyContextData(relevantFiles);
  }
}
```

Decision Points:
- [ ] Parallel or sequential file loading?
- [ ] Lazy loading (load on-demand)?
- [ ] Streaming for large files?
- [ ] Add progress callback for long loads?

🟡 MEDIUM PRIORITY (Verdent can implement, Claude Code should review)

These aspects should be implemented by Verdent, but reviewed by Claude Code after completion.

3. Content-Based Routing (PROMPT 9)

What to Review:

✅ **Classification logic:**
- How to determine agent type from event content?
- How to extract task type?
- What if event belongs to multiple categories?

✅ **Edge cases:**
- Ambiguous events (could go to multiple directories)
- Events with no clear classification
- Events that should go to multiple directories

Review After Implementation:
- [ ] Classification accuracy (manual testing)
- [ ] Edge case handling
- [ ] Performance (classification speed)

4. Interestingness Scoring (PROMPT 10)

What to Review:

✅ **Scoring algorithm:**
- What makes an event “interesting”?
- How to combine multiple indicators?
- Threshold tuning (too high = miss learning, too low = noise)

✅ **Learning indicators:**
- How to detect decisions vs regular events?
- How to detect breakthroughs vs incremental progress?
- How to detect patterns?

Review After Implementation:
- [ ] Scoring accuracy (manual review of scored events)
- [ ] Threshold tuning (adjust based on results)
- [ ] False positives/negatives

🟢 LOW PRIORITY (Verdent can handle)

5. Hydration Strategy Registry (PROMPT 12)

What to Review:

✅ **Strategy definitions:**
- What context to load for each strategy?
- How to parameterize strategies?

✅ **Registry pattern:**
- How to register strategies?
- How to retrieve strategies?

Review After Implementation:
- [ ] Strategy effectiveness (does each strategy load the right context?)
- [ ] Strategy coverage (do we have strategies for all task types?)

Review Workflow

After PROMPT 8 (Hook System):
1. ✅ Run quality gates: `pnpm check:all`
2. ✅ Verify all tests passing
3. 🔴 **MARK FOR CLAUDE CODE REVIEW**
4. 🔴 **Do not proceed to PROMPT 9 until reviewed**

After PROMPT 11 (Dynamic Context Loading):
1. ✅ Run quality gates: `pnpm check:all`
2. ✅ Verify all tests passing
3. 🔴 **MARK FOR CLAUDE CODE REVIEW**
4. 🔴 **Do not proceed to PROMPT 12 until reviewed**

After PROMPT 9, 10, 12:
1. ✅ Run quality gates: `pnpm check:all`
2. ✅ Verify all tests passing
3. 🟡 **Mark for post-Phase 2 review** (not blocking)

Part 7: Success Criteria for Phase 2

Overall Phase 2 Success Criteria

Functional Requirements:
✅ Hook system implemented (EventEmitter + 4 handlers)
✅ Content router implemented (routes to correct directories)
✅ Interestingness scorer implemented (tags high-signal events)
✅ Dynamic context loader implemented (AI-driven)
✅ Hydration strategy registry implemented (5 named strategies)

Quality Requirements:
✅ All tests passing (target: **550+ tests**, current: 472 Phase 1)
✅ Coverage maintained **>85%** (target: **>88%**)
✅ No ESLint violations
✅ All files formatted with Prettier
✅ TypeScript strict mode enabled

Integration Requirements:
✅ Integration with Phase 1 working (uses FileOperations, DirectoryOperations, etc.)
✅ All Phase 1 tests still passing
✅ No regressions in Phase 1 functionality

Documentation Requirements:
✅ API documentation updated
✅ Architecture diagrams updated
✅ Living Hydration doc updated
✅ Verification procedures updated

Per-Prompt Success Criteria

PROMPT 8: Hook System

Functional:
✅ EventEmitter class created
✅ Event interface and EventType enum defined
✅ HookHandler base class created
✅ 4 hook handlers created:
- `capture-all-handler.ts`
- `stop-handler.ts`
- `subagent-stop-handler.ts`
- `session-summary-handler.ts`

✅ Events captured and written to memory

Quality:
✅ Tests passing (**30+ tests**)
✅ Coverage **>85%** for `src/hooks/`
✅ Integration tests with Phase 1 FileOperations
✅ No ESLint violations in `src/hooks/`

Integration:
✅ Uses `FileOperations.appendFile()` for event capture
✅ Uses `DirectoryOperations.ensureDirectory()` for directory creation
✅ Uses `SecurityAuditLogger.logSecurityEvent()` for audit logs

Documentation:
✅ Hook system architecture documented
✅ Event schema documented
✅ Hook handler API documented

PROMPT 9: Content-Based Routing

Functional:
✅ ContentRouter class created
✅ Classification logic implemented:
- Extract agent type
- Extract task type
- Extract project context
- Extract tags
✅ Events routed to correct directories:
- Agent events → `.ufc/agents/{agent_id}/history/`
- Project events → `.ufc/projects/{project_id}/history/`
- Session events → `.ufc/sessions/current/history/`

Quality:
✅ Tests passing (**25+ tests**)
✅ Coverage **>85%** for `src/routing/content-router.ts`
✅ Integration tests with Phase 1 DirectoryOperations
✅ No ESLint violations in `src/routing/`

Integration:
✅ Uses `DirectoryOperations.ensureDirectory()` for directory creation
✅ Uses `PathValidator.validate()` for path validation
✅ Uses `GuardrailValidator.validate()` for guardrail checks

Documentation:
✅ Content routing logic documented
✅ Classification algorithm documented
✅ Routing rules documented

PROMPT 10: Interestingness Scoring

Functional:
✅ InterestingnessScorer class created

✅ Scoring logic implemented:
- Decisions (0.7)
- Breakthroughs (0.9)
- Failures (0.8)
- Patterns (0.6)
- Edge cases (0.7)
✅ High-signal events tagged with interestingness score
✅ Events promoted to learned/ when score >= threshold

Quality:
✅ Tests passing (**20+ tests**)
✅ Coverage **>85%** for `src/routing/interestingness-scorer.ts`
✅ Integration tests with Phase 1 FileOperations
✅ No ESLint violations in `src/routing/`

Integration:
✅ Uses `FileOperations.writeFile()` for learned items
✅ Uses `FileNamingConvention.generate()` for filenames
✅ Uses `SecurityAuditLogger.logSecurityEvent()` for promotions

Documentation:
✅ Scoring algorithm documented
✅ Learning indicators documented
✅ Promotion logic documented

PROMPT 11: Dynamic Context Loading

Functional:
✅ DynamicContextLoader class created
✅ AI-driven context selection working:
- LLM API integration (Anthropic Claude)
- Prompt engineering for context selection
- Response parsing
✅ Relevance scoring implemented
✅ 4-layer context loading working:
- User layer → `.ufc/user/`
- Project layer → `.ufc/projects/{project_id}/`
- Session layer → `.ufc/sessions/current/`
- Agent layer → `.ufc/agents/{agent_id}/`
✅ Context caching implemented

Quality:
✅ Tests passing (**30+ tests**)
✅ Coverage **>85%** for `src/context/`
✅ Integration tests with Phase 1 FileOperations
✅ Mock LLM responses for testing
✅ No ESLint violations in `src/context/`

Integration:
✅ Uses `FileOperations.readFile()` for context files

✅ Uses `PathValidator.validate()` for path validation
✅ Uses `DirectoryOperations.ensureDirectory()` for directory creation

Documentation:
✅ Dynamic context loading architecture documented
✅ AI-driven selection algorithm documented
✅ Caching strategy documented
✅ Performance optimization documented

PROMPT 12: Hydration Strategy Registry

Functional:
✅ HydrationStrategyRegistry class created
✅ 5 named strategies defined:
- `deep_research`
- `quick_fix`
- `build_spec`
- `code_review`
- `exploration`
✅ Strategy selection logic implemented
✅ Strategy execution working

Quality:
✅ Tests passing (**15+ tests**)
✅ Coverage **>85%** for `src/context/hydration-strategy-registry.ts`
✅ Integration tests with DynamicContextLoader
✅ No ESLint violations in `src/context/`

Integration:
✅ Integrates with DynamicContextLoader
✅ Uses Phase 1 APIs through DynamicContextLoader

Documentation:
✅ Strategy registry architecture documented
✅ Strategy definitions documented
✅ Strategy selection logic documented

Phase 2 Completion Checklist

Before marking Phase 2 complete:

- [ ] All 5 prompts completed (PROMPT 8-12)
- [ ] All functional requirements met
- [ ] All quality requirements met
- [ ] All integration requirements met
- [ ] All documentation requirements met
- [ ] Quality gates passing: `pnpm check:all`
- [ ] Tests passing: **550+ tests**
- [ ] Coverage: **>88%**

- [ ] Living Hydration doc updated
- [ ] Phase 2 completion report created
- [ ] Critical aspects reviewed by Claude Code (PROMPT 8, 11)

Phase 2 Completion Report includes:
- Summary of what was built
- Key metrics (tests, coverage, etc.)
- Known issues/limitations
- Recommendations for Phase 3
- Updated project timeline

Part 8: Technical Specifications

Dependencies to Add (Phase 2)

For Dynamic Context Loading (PROMPT 11):

```
# Install Anthropic SDK for LLM API
pnpm add @anthropic-ai/sdk

# Install dotenv for environment variables
pnpm add dotenv

# Install type definitions
pnpm add -D @types/node
```

Environment Variables (.env):

```
# Anthropic API Key (for dynamic context loading)
ANTHROPIC_API_KEY=sk-ant-...

# Optional: Model configuration
ANTHROPIC_MODEL=claude-opus-4-20250514
ANTHROPIC_MAX_TOKENS=1024

# Optional: Context loading configuration
CONTEXT_CACHE_TTL=60000
CONTEXT_CACHE_SIZE=100
CONTEXT_RELEVANCE_THRESHOLD=0.5
```

Update package.json:

```
{
  "dependencies": {
    "@anthropic-ai/sdk": "^0.20.0",
    "dotenv": "^16.4.0"
  },
  "devDependencies": {
    "@types/node": "^20.11.0"
  }
}
```

Directory Structure (Phase 2)

New directories to create:

```
src/
├── hooks/                          # PROMPT 8
│   ├── index.ts                    # Exports all hook classes
│   ├── types.ts                    # Event, EventType, EventMetadata
│   ├── event-emitter.ts            # EventEmitter class
│   ├── hook-handler.ts             # HookHandler base class
│   ├── capture-all-handler.ts      # CaptureAllHandler
│   ├── stop-handler.ts             # StopHandler
│   ├── subagent-stop-handler.ts    # SubagentStopHandler
│   └── session-summary-handler.ts  # SessionSummaryHandler
├── context/                        # PROMPT 11, 12
│   ├── index.ts                    # Exports all context classes
│   ├── types.ts                    # ContextData, TaskContext, ContextFile
│   ├── dynamic-context-loader.ts   # DynamicContextLoader class
│   └── hydration-strategy-registry.ts  # HydrationStrategyRegistry + strategies
├── routing/                        # PROMPT 9, 10
│   ├── index.ts                    # Exports all routing classes
│   ├── types.ts                    # Classification, RoutingConfig
│   ├── content-router.ts           # ContentRouter class
│   └── interestingness-scorer.ts   # InterestingnessScorer class
tests/
├── hooks/                          # PROMPT 8
│   ├── event-emitter.test.ts
│   ├── hook-handler.test.ts
│   ├── capture-all-handler.test.ts
│   ├── stop-handler.test.ts
│   ├── subagent-stop-handler.test.ts
│   └── session-summary-handler.test.ts
├── context/                        # PROMPT 11, 12
│   ├── dynamic-context-loader.test.ts
│   └── hydration-strategy-registry.test.ts
└── routing/                        # PROMPT 9, 10
    ├── content-router.test.ts
    └── interestingness-scorer.test.ts
```

Phase 1 directories (unchanged):

```
src/
├── exceptions/
├── guardrails/
└── memory/
```

Event Schema (JSONL)

Event Interface:

```typescript
// src/hooks/types.ts

/**
 * Event interface for all events in Infinite Aura
 */
export interface Event {
  /** ISO 8601 timestamp */
  timestamp: string;

  /** Event type */
  type: EventType;

  /** Event content (text or JSON string) */
  content: string;

  /** Event metadata */
  metadata: EventMetadata;
}

/**
 * Event type enum
 */
export enum EventType {
  /** Capture all events */
  CAPTURE_ALL = 'capture_all',

  /** Stop event (task completed) */
  STOP = 'stop',

  /** Subagent stop event (subagent completed) */
  SUBAGENT_STOP = 'subagent_stop',

  /** Session summary event */
  SESSION_SUMMARY = 'session_summary',
}

/**
 * Event metadata interface
 */
export interface EventMetadata {
  /** Agent ID (if applicable) */
  agentId?: string;

  /** Project ID (if applicable) */
  projectId?: string;

  /** Session ID (if applicable) */
  sessionId?: string;

  /** Tags for event classification */
  tags?: string[];

  /** Interestingness score (0-1) */
  interestingness?: number;

  /** Event description (for file naming) */
  description?: string;

  /** Additional metadata (extensible) */
  [key: string]: any;
}
```

Example Events (JSONL):


```
{"timestamp":"2026-01-06T12:00:00Z","type":"capture_all","content":"User started new research task","metadata":{"agentId":"deep_research","projectId":"proj_123","sessionId":"sess_456","tags":["research","start"]}}
{"timestamp":"2026-01-06T12:05:00Z","type":"capture_all","content":"Agent identified key insight","metadata":{"agentId":"deep_research","projectId":"proj_123","sessionId":"sess_456","tags":["research","insight"],"interestingness":0.85,"description":"authentication_pattern"}}
{"timestamp":"2026-01-06T12:10:00Z","type":"stop","content":"Research task completed","metadata":{"agentId":"deep_research","projectId":"proj_123","sessionId":"sess_456","reason":"success","duration":600}}
{"timestamp":"2026-01-06T12:10:01Z","type":"session_summary","content":"Completed research on authentication patterns. Key findings: ...","metadata":{"sessionId":"sess_456","projectId":"proj_123","eventCount":15,"duration":600}}
```

Context Schema

Context Interfaces:

```typescript
// src/context/types.ts

/**
 * Context data interface (4-layer context)
 */
export interface ContextData {
  /** User layer context */
  user: UserContext | null;

  /** Project layer context */
  project: ProjectContext | null;

  /** Session layer context */
  session: SessionContext | null;

  /** Agent layer context */
  agent: AgentContext | null;
}

/**
 * Task context for dynamic loading
 */
export interface TaskContext {
  /** Task description */
  description: string;

  /** Task type (research, debug, code_review, etc.) */
  type: string;

  /** Agent ID */
  agentId?: string;

  /** Project ID */
  projectId?: string;

  /** Session ID */
  sessionId?: string;

  /** Context depth (minimal, standard, extensive) */
  contextDepth?: 'minimal' | 'standard' | 'extensive';

  /** Include historical context */
  includeHistorical?: boolean;

  /** Include patterns */
  includePatterns?: boolean;
}

/**
 * Context file interface
 */
export interface ContextFile {
  /** File path */
  path: string;

  /** Context layer (user, project, session, agent) */
  layer: 'user' | 'project' | 'session' | 'agent';

  /** File description */
  description?: string;

  /** Relevance score (0-1) */
```

```
  relevance?: number;
}

/**
 * User context interface
 */
export interface UserContext {
  /** User ID */
  userId: string;

  /** User preferences */
  preferences: Record<string, any>;

  /** User profile */
  profile: Record<string, any>;
}

/**
 * Project context interface
 */
export interface ProjectContext {
  /** Project ID */
  projectId: string;

  /** Project metadata */
  metadata: Record<string, any>;

  /** Project files */
  files: string[];
}

/**
 * Session context interface
 */
export interface SessionContext {
  /** Session ID */
  sessionId: string;

  /** Session start time */
  startTime: string;

  /** Session events */
  events: Event[];
}

/**
 * Agent context interface
 */
export interface AgentContext {
  /** Agent ID */
  agentId: string;

  /** Agent configuration */
  config: Record<string, any>;

  /** Learned patterns */
  learned: any[];
}
```

Routing Schema

Classification Interface:

```
// src/routing/types.ts

/**
 * Event classification
 */
export interface Classification {
  /** Agent type (deep_research, code_review, etc.) */
  agentType: string | null;

  /** Task type (research, debug, code_review, etc.) */
  taskType: string | null;

  /** Project context (project ID) */
  projectContext: string | null;

  /** Tags for event */
  tags: string[];
}

/**
 * Routing configuration
 */
export interface RoutingConfig {
  /** Base path for UFC directory */
  basePath: string;

  /** Routing rules (agent type -> directory) */
  rules: Map<string, string>;

  /** Fallback directory */
  fallback: string;
}
```

Hydration Strategy Schema

Strategy Interface:

```typescript
// src/context/types.ts

/**
 * Hydration strategy interface
 */
export interface HydrationStrategy {
  /** Strategy name */
  name: string;

  /** Strategy description */
  description: string;

  /**
   * Execute strategy
   * @param loader - Dynamic context loader
   * @param taskContext - Task context
   * @returns Context data
   */
  execute(
    loader: DynamicContextLoader,
    taskContext: TaskContext
  ): Promise<ContextData>;
}
```

Example Strategies:

```
// Deep research strategy
export class DeepResearchStrategy implements HydrationStrategy {
  name = 'deep_research';
  description = 'Load extensive context for deep research tasks';

  async execute(
    loader: DynamicContextLoader,
    taskContext: TaskContext
  ): Promise<ContextData> {
    // Override context settings
    taskContext.contextDepth = 'extensive';
    taskContext.includeHistorical = true;
    taskContext.includePatterns = true;

    return await loader.loadContext(taskContext);
  }
}

// Quick fix strategy
export class QuickFixStrategy implements HydrationStrategy {
  name = 'quick_fix';
  description = 'Load minimal context for quick fixes';

  async execute(
    loader: DynamicContextLoader,
    taskContext: TaskContext
  ): Promise<ContextData> {
    // Override context settings
    taskContext.contextDepth = 'minimal';
    taskContext.includeHistorical = false;
    taskContext.includePatterns = false;

    return await loader.loadContext(taskContext);
  }
}
```

Part 9: Workflow for Verdent

Step-by-Step Workflow

Before Starting Phase 2:

1. ✅ **Read this handoff document completely**
   - Understand all 10 parts
   - Review prompt sequence (PROMPT 8-12)
   - Note critical aspects for Claude Code review

2. ✅ **Review Phase 1 code**
   - Read `src/exceptions/` - Exception hierarchy
   - Read `src/guardrails/` - Guardrail policies
   - Read `src/memory/` - Memory scaffold APIs
   - Understand UFC directory structure (`.ufc/`)

3. ✅ **Install Phase 2 dependencies**
   `bash`

```
pnpm add @anthropic-ai/sdk dotenv
pnpm add -D @types/node
```

4. ✅ **Create .env file**

```
bash
# Create .env file
echo "ANTHROPIC_API_KEY=your_api_key_here" > .env
echo "ANTHROPIC_MODEL=claude-opus-4-20250514" >> .env
```

5. ✅ **Verify Phase 1 still working**

```
bash
pnpm check:all
# Should show: 472 tests passing, 90.05% coverage
```

During Phase 2:

PROMPT 8: Hook System Foundation (2-3 hours)

Step 1: Create hook types

```
# Create hooks directory
mkdir -p src/hooks

# Create types file
touch src/hooks/types.ts
```

What to implement:
- [ ] Event interface
- [ ] EventType enum
- [ ] EventMetadata interface

Step 2: Create EventEmitter

```
touch src/hooks/event-emitter.ts
```

What to implement:
- [ ] EventEmitter class
- [ ] registerHandler() method
- [ ] unregisterHandler() method
- [ ] emit() method
- [ ] Error handling (Promise.allSettled)

Step 3: Create HookHandler base class

```
touch src/hooks/hook-handler.ts
```

What to implement:
- [ ] Abstract HookHandler class
- [ ] handle() abstract method

- [ ] getEventFilePath() helper method
- [ ] validate() helper method

Step 4: Create 4 hook handlers

```
touch src/hooks/capture-all-handler.ts
touch src/hooks/stop-handler.ts
touch src/hooks/subagent-stop-handler.ts
touch src/hooks/session-summary-handler.ts
```

What to implement:
- [ ] CaptureAllHandler - Append all events to history
- [ ] StopHandler - Write stop events with reason
- [ ] SubagentStopHandler - Write subagent completion events
- [ ] SessionSummaryHandler - Generate session summaries

Step 5: Create tests

```
mkdir -p tests/hooks
touch tests/hooks/event-emitter.test.ts
touch tests/hooks/hook-handler.test.ts
touch tests/hooks/capture-all-handler.test.ts
touch tests/hooks/stop-handler.test.ts
touch tests/hooks/subagent-stop-handler.test.ts
touch tests/hooks/session-summary-handler.test.ts
```

What to test:
- [ ] EventEmitter: register, emit, error handling
- [ ] HookHandler: base class functionality
- [ ] Each handler: event capture, file writing
- [ ] Integration: EventEmitter + handlers + FileOperations

Step 6: Run quality gates

```
pnpm test              # Run tests
pnpm coverage          # Check coverage
pnpm lint              # Check ESLint
pnpm format            # Format with Prettier
pnpm check:all         # Run all quality gates
```

Step 7: Mark for Claude Code review
- 🔴 Do not proceed to PROMPT 9 until reviewed
- 🔴 Address all review feedback before continuing

PROMPT 9: Content-Based Routing (2-3 hours)

Step 1: Create routing types

```
mkdir -p src/routing
touch src/routing/types.ts
```

What to implement:
- [ ] Classification interface
- [ ] RoutingConfig interface

Step 2: Create ContentRouter

```
touch src/routing/content-router.ts
```

What to implement:
- [ ] ContentRouter class
- [ ] classifyEvent() method
- [ ] routeEvent() method
- [ ] extractAgentType() helper
- [ ] extractTaskType() helper
- [ ] extractProjectContext() helper
- [ ] extractTags() helper

Step 3: Create tests

```
mkdir -p tests/routing
touch tests/routing/content-router.test.ts
```

What to test:
- [ ] Classification logic for different event types
- [ ] Routing to correct directories
- [ ] Edge cases (ambiguous events)
- [ ] Integration with Phase 1 DirectoryOperations

Step 4: Run quality gates

```
pnpm check:all
```

Step 5: Mark for post-Phase 2 review
- 🟡 Not blocking, but should be reviewed

PROMPT 10: Interestingness Scoring (2-3 hours)

Step 1: Create InterestingnessScorer

```
touch src/routing/interestingness-scorer.ts
```

What to implement:
- [ ] InterestingnessScorer class
- [ ] scoreEvent() method
- [ ] promoteToLearned() method
- [ ] isDecision() helper
- [ ] isBreakthrough() helper
- [ ] isFailure() helper

- [ ] isPattern() helper
- [ ] isEdgeCase() helper

Step 2: Create tests

```
touch tests/routing/interestingness-scorer.test.ts
```

What to test:
- [ ] Scoring logic for each indicator
- [ ] Score calculation
- [ ] Promotion to learned/
- [ ] Integration with Phase 1 FileOperations

Step 3: Run quality gates

```
pnpm check:all
```

Step 4: Mark for post-Phase 2 review
- 🟡 Not blocking, but should be reviewed

PROMPT 11: Dynamic Context Loading (3-5 hours)

Step 1: Create context types

```
mkdir -p src/context
touch src/context/types.ts
```

What to implement:
- [ ] ContextData interface
- [ ] TaskContext interface
- [ ] ContextFile interface
- [ ] UserContext, ProjectContext, SessionContext, AgentContext

Step 2: Create DynamicContextLoader

```
touch src/context/dynamic-context-loader.ts
```

What to implement:
- [ ] DynamicContextLoader class
- [ ] loadContext() method
- [ ] selectRelevantFiles() method (uses LLM)
- [ ] listAllContextFiles() method
- [ ] scoreRelevance() method
- [ ] Cache implementation
- [ ] LLM API integration (Anthropic)

Step 3: Create tests

```
mkdir -p tests/context
touch tests/context/dynamic-context-loader.test.ts
```

What to test:
- [ ] Context selection logic
- [ ] Relevance scoring
- [ ] 4-layer context loading
- [ ] Cache functionality
- [ ] LLM integration (with mocks)
- [ ] Integration with Phase 1 FileOperations

Step 4: Run quality gates

```
pnpm check:all
```

Step 5: Mark for Claude Code review
- 🔴 Do not proceed to PROMPT 12 until reviewed
- 🔴 Address all review feedback before continuing

PROMPT 12: Hydration Strategy Registry (2-3 hours)

Step 1: Create HydrationStrategyRegistry

```
touch src/context/hydration-strategy-registry.ts
```

What to implement:
- [ ] HydrationStrategy interface
- [ ] HydrationStrategyRegistry class
- [ ] 5 strategy classes:
- DeepResearchStrategy
- QuickFixStrategy
- BuildSpecStrategy
- CodeReviewStrategy
- ExplorationStrategy

Step 2: Create tests

```
touch tests/context/hydration-strategy-registry.test.ts
```

What to test:
- [ ] Strategy registration
- [ ] Strategy selection
- [ ] Strategy execution
- [ ] Integration with DynamicContextLoader

Step 3: Run quality gates

```
pnpm check:all
```

Step 4: Mark for post-Phase 2 review
- 🟢 Low priority, can be reviewed after Phase 2 complete

After Each Prompt:

1. ✅ **Run quality gates**
   ```
   bash
   pnpm check:all
   ```
2. ✅ **Verify tests passing**
   - Check test count (should increase)
   - Check coverage (should be >85%)
   - Check for any regressions
3. ✅ **Update Living Hydration doc**
   - Document what was implemented
   - Add any new patterns/decisions
   - Update progress tracking
4. ✅ **Commit changes**
   ```
   bash
   git add .
   git commit -m "Phase 2: Implement [component]"
   ```
5. ✅ **Mark for review (if critical)**
   - PROMPT 8, 11: 🔴 HIGH - Mark for Claude Code review
   - PROMPT 9, 10, 12: 🟡 MEDIUM or 🟢 LOW - Continue

After Phase 2 Complete:

1. ✅ **Run full verification**
   ```
   bash
   pnpm check:all
   # Verify: 550+ tests, >88% coverage, all gates passing
   ```
2. ✅ **Update all documentation**
   - API documentation
   - Architecture diagrams
   - Living Hydration doc
   - Verification procedures
3. ✅ **Create Phase 2 completion report**
   ```
   bash
   touch ~/infinite_aura_phase2_completion.md
   ```

Report should include:
- Summary of what was built (5 capabilities)

- Key metrics (tests: 550+, coverage: >88%)
- Integration with Phase 1 (verified)
- Known issues/limitations
- Recommendations for Phase 3
- Updated project timeline

1. ✅ **Commit Phase 2 completion**
   `bash`
   `git add .`
   `git commit -m "Phase 2: Complete - Hook System + Dynamic Context Loading"`
   `git tag phase-2-complete`

Quality Gates Reference

Run before each commit:

```
pnpm check:all
```

What it runs:
- TypeScript compilation (`pnpm build`)
- ESLint (`pnpm lint`)
- Prettier (`pnpm format:check`)
- Tests (`pnpm test`)
- Coverage (`pnpm coverage`)

Expected results (Phase 2 complete):
- ✅ TypeScript: No errors
- ✅ ESLint: No violations
- ✅ Prettier: All files formatted
- ✅ Tests: **550+/550+ passing**
- ✅ Coverage: **>88%**

If any quality gate fails:
1. Fix the issue
2. Re-run quality gates
3. Only proceed when all gates pass

Part 10: Next Steps

Immediate Next Steps

Right now (before starting PROMPT 8):

1. ✅ **Review this handoff document**
   - Read all 10 parts carefully
   - Understand the prompt sequence
   - Note critical aspects for review

2. ✅ **Review Phase 1 code**
   - `src/exceptions/` - Exception hierarchy
   - `src/guardrails/` - Guardrail policies
   - `src/memory/` - Memory scaffold APIs
   - Understand UFC directory structure

3. ✅ **Install Phase 2 dependencies**
   ```bash
   pnpm add @anthropic-ai/sdk dotenv
   pnpm add -D @types/node
   ```

4. ✅ **Create .env file**
   ```bash
   echo "ANTHROPIC_API_KEY=your_api_key_here" > .env
   echo "ANTHROPIC_MODEL=claude-opus-4-20250514" >> .env
   ```

5. ✅ **Verify Phase 1 still working**
   ```bash
   pnpm check:all
   # Expected: 472 tests passing, 90.05% coverage
   ```

6. ✅ **Prepare for PROMPT 8**
   - Create `src/hooks/` directory
   - Review PAI hook patterns (Part 4)
   - Review event schema (Part 8)

After Phase 2 Complete

Phase 3: CLI Interface + Persona Model (4-6 hours)

Goal: Build command-line interface and persona configuration

Capabilities:
- CLI commands (init, capture, summary, status)
- Persona model (user preferences, communication style)
- Configuration management
- Interactive prompts

Deliverables:
- CLI interface (using Commander.js or Yargs)
- Persona configuration
- Tests (50+ tests)
- CLI documentation

Estimated Duration: 4-6 hours (2 prompts: PROMPT 13-14)

Phase 4: Orchestrator + Agent Routing (16-24 hours)

Goal: Build orchestrator for multi-agent coordination

Capabilities:
- Agent orchestrator (manages multiple agents)
- Agent routing (route tasks to correct agents)
- Task queue management
- Agent lifecycle management

Deliverables:
- Orchestrator architecture
- Agent registry
- Task queue
- Tests (100+ tests)
- Orchestrator documentation

Estimated Duration: 16-24 hours (5 prompts: PROMPT 15-19)

Phase 5: Observability Dashboard (12-16 hours)

Goal: Build real-time observability dashboard

Capabilities:
- Real-time event monitoring
- Agent performance metrics
- Context visualization
- System health dashboard

Deliverables:
- Web dashboard (React + D3.js)
- Real-time event stream (WebSockets)
- Metrics aggregation
- Tests (60+ tests)
- Dashboard documentation

Estimated Duration: 12-16 hours (4 prompts: PROMPT 20-23)

Phase 6: Meta-Learning + Proactive Triggers (12-18 hours)

Goal: Build meta-learning system for continuous improvement

Capabilities:
- Pattern detection (identify recurring patterns)
- Proactive triggers (suggest actions before user asks)
- Meta-learning (learn from all interactions)
- Self-improvement (optimize context loading, routing, etc.)

Deliverables:
- Pattern detection engine
- Proactive trigger system
- Meta-learning algorithms
- Tests (80+ tests)
- Meta-learning documentation

Estimated Duration: 12-18 hours (4 prompts: PROMPT 24-27)

Project Timeline

Phase 1: ✅ **Complete** (Foundation)
- Duration: 16 hours
- Tests: 472
- Coverage: 90.05%
- Status: ✅ Complete

Phase 2: 🚀 **Starting Now** (Hook System + Dynamic Context)
- Duration: 12-18 hours
- Tests: +120 (total: 550+)
- Coverage: >88%
- Status: 🚀 Ready to start

Phase 3: ⏳ **Next** (CLI + Persona)
- Duration: 4-6 hours
- Tests: +50 (total: 600+)
- Coverage: >88%
- Status: ⏳ Pending Phase 2

Phase 4: ⏳ **Future** (Orchestrator + Agent Routing)
- Duration: 16-24 hours
- Tests: +100 (total: 700+)
- Coverage: >88%
- Status: ⏳ Pending Phase 3

Phase 5: ⏳ **Future** (Observability Dashboard)
- Duration: 12-16 hours
- Tests: +60 (total: 760+)
- Coverage: >88%
- Status: ⏳ Pending Phase 4

Phase 6: ⏳ **Future** (Meta-Learning)
- Duration: 12-18 hours
- Tests: +80 (total: 840+)
- Coverage: >88%
- Status: ⏳ Pending Phase 5

Total Project Timeline: 72-98 hours (9-12 weeks at 8 hours/week)

Success Criteria (Overall Project)

When is Infinite Aura complete?

✅ All 6 phases complete
✅ 840+ tests passing
✅ Coverage >88%
✅ All quality gates passing

✅ Complete documentation
✅ Production-ready codebase
✅ Deployed and operational

Key capabilities working:
✅ Memory scaffold (Phase 1)
✅ Hook system (Phase 2)
✅ Dynamic context loading (Phase 2)
✅ CLI interface (Phase 3)
✅ Multi-agent orchestration (Phase 4)
✅ Observability dashboard (Phase 5)
✅ Meta-learning (Phase 6)

Communication Protocol

How to communicate with Claude Code (me):

For Critical Reviews (PROMPT 8, 11):
1. Complete the prompt
2. Run quality gates (`pnpm check:all`)
3. Create review request:
- Summary of what was implemented
- Any design decisions made
- Any concerns/questions
- Request for review
4. Wait for review before proceeding

For Medium/Low Priority (PROMPT 9, 10, 12):
1. Complete the prompt
2. Run quality gates
3. Continue to next prompt
4. Mark for post-Phase 2 review

For General Questions:
- Ask anytime during implementation
- Include context (which prompt, what you're implementing)
- Include code snippets if relevant

For Phase Completion:
- Create Phase 2 completion report
- Include all metrics (tests, coverage, etc.)
- Request final review before Phase 3

Conclusion

This handoff document provides everything Verdent Desktop needs to complete Phase 2:

✅ **Phase 1 Summary** - What was built and what's available
✅ **Phase 2 Overview** - What to build and why

✅ **Integration Points** - How Phase 2 connects to Phase 1
✅ **PAI Patterns** - Proven patterns to adapt
✅ **Prompt Sequence** - Step-by-step guide (PROMPT 8-12)
✅ **Critical Aspects** - What needs Claude Code review
✅ **Success Criteria** - How to know when Phase 2 is complete
✅ **Technical Specs** - Schemas, interfaces, dependencies
✅ **Workflow** - Detailed step-by-step process
✅ **Next Steps** - What comes after Phase 2

Remember:
- 🔴 HIGH priority (PROMPT 8, 11): Must review with Claude Code
- 🟡 MEDIUM priority (PROMPT 9, 10): Implement and mark for review
- 🟢 LOW priority (PROMPT 12): Implement and continue

Quality is paramount:
- Always run `pnpm check:all` before committing
- Maintain >85% coverage (target: >88%)
- Write tests as you implement (not after)
- Update documentation as you go

Good luck with Phase 2! 🚀

Questions? Need clarification? Ask Claude Code anytime.